PRODUCT REQUIREMENTS DOCUMENT

T-Pay

Product Requirements Document

Offline-first payments on a dual-device trust model

Internal — baseline for V1 build. This document records what is built, what is simulated, and what the system must do next. Section 1 is the register; section 3 is the specification.

Version 1.0 | 4 November 2026 | T-Pay engineering

Contents

1. Where we are

- 1.1 How to read the register
- 1.2 Domain logic
- 1.3 Device identity and cryptography
- 1.4 Vault and double-spend
- 1.5 Settlement
- 1.6 Sync and dual-device ledger
- 1.7 Applications and backend
- 1.8 Assistant and non-financial requirements
- 1.9 Documentation
- 1.10 Reality check
- 1.11 Not building now

2. Requirement gaps

- 2.1 Functional gaps
- 2.2 Non-functional gaps
- 2.3 Asked for by the source document, absent from the code
- 2.4 Baseline defects carried on the main branch
- 2.5 What the source material leaves open

3. The PRD

- 3.1 Product definition
- 3.2 Actors
- 3.3 Scope
- 3.4 Functional requirements
- 3.5 Non-functional requirements
- 3.6 Transaction lifecycle
- 3.7 Minimum interface surface
- 3.8 Definition of done for V1

4. PayPal constraints as requirements

4.1 What PayPal will and will not do

4.2 Requirements

4.3 Consequences the product must accept

4.4 Sources

5. Device trust

5.1 Two assurance tiers

5.2 Signing rules

5.3 Why the layering matters

5.4 Sources

6. Architecture contract

6.1 Rules

6.2 Bounded context ownership

6.3 Import matrix

6.4 Review checklist

6.5 Enforcement

7. Roadmap

7.1 Parked

8. Open inputs and decisions

Appendix A. Evidence index

Appendix B. Trace from the source document

1. Where we are

This section is a register, not a narrative. Every capability the product needs is an item with an identifier, a status, the evidence that justifies that status, and the next action. Nothing here is aspirational, and no status is granted on the strength of a comment, a README claim, or a class name.

1.1 How to read the register

STATUS	MEANING
Met	Implemented and exercised by either an executable check or a reachable UI path.
Simulated	A real interface exists, but the behaviour behind it is a stand-in that does not deliver the guarantee it implies.
Partial	Part of the requirement is real, part is absent or unreachable in the shipped surface.
Missing	No implementation exists.

Evidence paths are relative to the repository root. The status vocabulary is enforced by rule AR-08 in section 6: a capability may be described as working only if something executable exercises it.

1.2 Domain logic

The domain layer is the most trustworthy part of the system. It was built first, it is independent of any framework, and it is the only area with an executable check behind it.

ID	ITEM	STATUS	EVIDENCE	NEXT ACTION
W-01	Shared kernel: entity, value object, aggregate root, identifier, result type, money, event bus	Met	`core/shared/domain/`	Keep as-is; add clock and random ports so aggregates stop reading the wall clock (AR-05).
W-02	Escrow aggregate, cryptogram, device identity, status value objects, repository and gateway ports	Met	`core/escrow/domain/`, `core/escrow/application/ports/`	Extend with voucher sequence and vault binding (W-12).
W-03	Merchant context: inventory aggregate, price-to-quantity derivation, spoken inventory parser, sale recording	Met	`core/resource-tracking/merchant/`	None for V1.
W-04	Budget context: plan, categories, categoriser service, ambiguity detection	Met	`core/resource-tracking/budget/`	None for V1.
W-05	Transit context: seven-seat trip aggregate, passenger seats, fare collection	Met	`core/resource-tracking/transit/`	None for V1.
W-06	Vault aggregate with duration, spend authorisation and melt	Met	`core/escrow/domain/MarketVault.ts`	Bind to PayPal authorisation and to vouchers (W-12).
W-07	Composition root and four-context wiring with an executable verification script	Met	`core/container.ts`, `core/__tests__/domain-verification.ts`	Promote the script into a real test runner during M9.

1.3 Device identity and cryptography

This is the area the source document leans on hardest and where the codebase is furthest from the requirement. The user-facing promise is that both devices hold an identity, sign every transaction, and can prove to each other who they are. None of that exists.

ID	ITEM	STATUS	EVIDENCE	NEXT ACTION
W-08	Device keypair, key custody, backend-issued device token, public-key registry	Missing	`DeviceIdentity` declares an optional `publicKey` that is never populated (`core/escrow/domain/value-objects/DeviceIdentity.ts`)	Generate an ECDSA P-256 keypair per device, register the public key, issue a device token (FR-01 to FR-03).
W-09	Voucher signatures and verification	Simulated	`core/escrow/domain/services/CryptographicExchangeService.ts` computes a 32-bit string hash and labels it `HMAC-SHA256` ; `verifyProof` only checks a string prefix; the nonce is generated and never inspected	Replace with a real signature over canonical bytes (FR-11, DT-04 to DT-06).
W-10	Canonical serialisation, sequence numbers, replay cache, high-water mark	Missing	No code path reads `Cryptogram.nonce`	Adopt JCS canonicalisation, per-device monotonic sequence, server high-water mark (FR-13, FR-14).

1.4 Vault and double-spend

ID	ITEM	STATUS	EVIDENCE	NEXT ACTION
W-11	Vault linked to offline spending	Partial	`MarketVault.authorizeSpend` is referenced only by `core/__tests__/domain-verification.ts` ; `CreateOfflineEscrowUseCase` never consults a vault, so vouchers can exceed the locked amount	Debit the vault when a voucher is signed, and refuse to sign beyond the remaining balance (FR-07, FR-08).
W-12	Vault as a real PayPal authorization, with reserved-versus-settled accounting and melt that respects in-flight vouchers	Missing	`MarketVault` fabricates a `paypalPreAuthId` from `Math.random()` ; `meltUnspentFunds` ignores unsettled vouchers	Map lock to an authorisation order, spend to partial capture, melt to void (PP-04, PP-05, FR-06, FR-09).

1.5 Settlement

ID	ITEM	STATUS	EVIDENCE	NEXT ACTION
W-13	Settlement service: server-side capture, idempotency, leasing, receipts, webhooks, reconciliation	Simulated	`PayPalSdkGatewayAdapter` runs in the browser, reads credentials from `localStorage` , and falls back to a mock capture that always succeeds (`core/escrow/infrastructure/adapters/PayPalSdkGatewayAdapter.ts`)	Move settlement behind the backend, own dedupe in the database, add webhook handlers (FR-17 to FR-21, PP-02 to PP-09).
W-14	Currency conversion layer	Missing	NGN converted to USD by an inline `/ 1500` constant inside the adapter	One FX port, rate snapshotted on the voucher, no constants in adapters (FR-10, PP-01).

1.6 Sync and dual-device ledger

Both devices are supposed to hold the same signed transaction, so that whichever reconnects first can settle it and both see the money move. Only the sender keeps a record today.

ID	ITEM	STATUS	EVIDENCE	NEXT ACTION
W-15	Receiver-side persistence and credit confirmation	Partial	`BroadcastChannelProximityAdapter.onTokenReceived` exists but no listener is ever registered in `core/container.ts`; only the sender persists the token	Persist received vouchers on the peer device and surface a pending-credit entry (FR-22, FR-23).
W-16	Sync engine: reconnect trigger, retry with backoff, receipt relay, vault-versus-ledger consistency check	Missing	`toggleNetwork` in `src/hooks/useTPayDomain.ts` calls one sync pass and sets a toast	Build the sync state machine and reconciliation pass (FR-24, FR-25).

1.7 Applications and backend

ID	ITEM	STATUS	EVIDENCE	NEXT ACTION
W-17	Web application as a real client of the domain	Partial	`src/components/PayPalEscrowModal.tsx` and `src/App.tsx` hold their own vault, token and balance state; `TrackerView` and `VoiceAssistantModal` do use the container, so there are two sources of truth and a duplicated intent parser	Remove local mirrors; one owner per fact (AR-03, AR-04).
W-18	React Native application on the shared domain	Partial	`react-native-app/App.tsx` is a 2,013-line screen file with its own hardcoded state and no import from `core/`	Consume the shared core, keep only rendering (M7).
W-19	Backend as a stateful service	Partial	Server-side controllers reuse `LocalStorage*Repository` classes that no-op in Node because `window` is undefined; one hardcoded user; no auth; no persistence	Introduce a real repository adapter and device authentication (M1, M2).
W-20	Typecheck and build hygiene	Missing	`npx tsc --noEmit` reports two errors: `cors` is imported but not a dependency, and `vault.controller.ts` omits `userId` required by `MeltMarketVaultDTO`	Fix both; make typecheck a gate (M0, NFR-11).

1.8 Assistant and non-financial requirements

ID	ITEM	STATUS	EVIDENCE	NEXT ACTION
W-21	Hybrid assistant: offline intent, online model fallback, dialect coverage, spoken readout	Simulated	`LocalSLMAdapter` is keyword matching; `executionMode: 'online_cloud'` is accepted and ignored in `ProcessSpokenCommandUseCase`; dialect handling is a string transform in `MultilingualVoiceService`	Label the offline path honestly, add the server-side fallback, move strings to a catalogue (FR-31 to FR-34).
W-22	Durability, observability, privacy, deployment, test pyramid, CI	Missing	Storage is `window.localStorage` only; logging is `console.warn`; no privacy or retention policy; no deploy target; one console-assertion script; no CI configuration	Address in M9 and NFR-01 to NFR-14.

1.9 Documentation

ID	ITEM	STATUS	EVIDENCE	NEXT ACTION
W-23	Documentation that matches the system	Simulated	`README.md` claims real HMAC signatures, a Bluetooth mesh, a secure enclave, a working PayPal integration and a fully passing suite; `DDD_ARCHITECTURE.md` describes an `OfflineEscrowToken` state machine with transitions the code does not implement	Replace claims with this document's C:5 register and keep them current (AR-08).

1.10 Reality check

Of twenty-three register items: **7 are met, 9 are simulated or partial, and 7 are missing entirely**. An earlier summary in this thread said fourteen items were not what the documentation claims; recounting item by item shows the true figure is sixteen, so the register — not the summary — is authoritative.

The shape of that result is consistent: everything that is purely business logic works, and everything that touches money movement, identity, or the guarantee that a voucher cannot be spent twice is either simulated or absent. That is precisely the half of the product that the PayPal integration and the offline claim stand on. Any pitch material written before W-08 to W-16 close should describe those flows as prototypes.

1.11 Not building now

Recorded so the register cannot quietly grow. Each has a reason, and each is re-evaluated only after V1 ships.

PARKED	WHY NOT NOW
B2B offline-protocol API for banks and fintechs	Requires a protocol standard, certification and a partner. It is a business decision that follows a working closed loop, not a feature.
Interoperability with Moniepoint, OPay, PalmPay	Depends on the protocol above.
Multiple currencies	The FX port is built for one display currency and one settlement currency. Adding pairs is a data problem that V1 does not need.
KYC and onboarding	Sandbox accounts only in V1; no real customer funds move.
Card issuance, savings, yield	Out of the product's stated problem.
Native BLE on web	Not possible in a browser. V1 uses the web as the second device through an explicit transport port; real radio work lands with M7.

2. Requirement gaps

Gaps are traced to register items so that nothing in this section can be closed by writing prose. Every gap below is closed by a named requirement in section 3 or 4.

2.1 Functional gaps

ID	GAP	ITEM	CLOSED BY
G-F01	Devices have no cryptographic identity, so a peer cannot verify who signed a voucher.	W-08	FR-01, FR-02, FR-03
G-F02	Signatures are not signatures: a 32-bit string hash presented as HMAC-SHA256, with prefix-only verification.	W-09	FR-11, DT-04
G-F03	Nonces are generated and never checked, so a captured voucher can be replayed indefinitely.	W-10	FR-13, FR-14, DT-07
G-F04	The vault does not constrain offline spending, so the double-spend the product exists to prevent is still open.	W-11	FR-07, FR-08
G-F05	The vault's PayPal identifiers are random strings; nothing is actually held on PayPal.	W-12	FR-06, PP-04
G-F06	Settlement runs in the client with a secret in browser storage and a mock fallback that always reports success.	W-13	FR-17, PP-02, PP-03
G-F07	No idempotency or lease, so two reconnecting devices can capture the same voucher twice.	W-13	FR-18, FR-19
G-F08	Naira is converted with an inline constant, and the rate is not recorded anywhere.	W-14	FR-10, PP-01
G-F09	The receiver never persists a voucher, so there is no dual-device ledger and no credit-side confirmation.	W-15	FR-22, FR-23
G-F10	There is no reconnect trigger, retry policy or reconciliation pass.	W-16	FR-24, FR-25
G-F11	The online assistant path is accepted and discarded, and dialect support is a string transform.	W-21	FR-31, FR-32, FR-33
G-F12	Web and mobile each maintain a private copy of domain state, with a duplicated intent parser.	W-17, W-18	FR-35, AR-03, AR-04

2.2 Non-functional gaps

ID	GAP	ITEM	CLOSED BY
G-N01	Storage is `localStorage`: 5 MB, synchronous, evictable, and silently lost when the origin is cleared. An offline payment that evaporates is worse than no payment.	W-22	NFR-01
G-N02	The PayPal client secret is written to browser `localStorage`.	W-13	NFR-02, PP-02
G-N03	No audit trail linking a settled amount to a PayPal capture identifier.	W-13	NFR-03, FR-20
G-N04	No idempotency and no duplicate-settlement test, so correctness under concurrency is unproven.	W-13, W-16	NFR-04
G-N05	No latency budget for the only interaction that must feel instant: a spoken sale confirmed offline.	W-21	NFR-05

ID	GAP	ITEM	CLOSED BY
G-N06	One console-assertion script, no runner, no CI gate, and two typecheck errors accepted on `main`.	W-20, W-22	NFR-11, NFR-12
G-N07	The domain cannot be shared by three surfaces as written: adapters read `window` and `navigator`, and repositories are browser classes.	W-19	NFR-13, AR-02
G-N08	No localisation catalogue, no behaviour when no speech voice is installed, no privacy or retention policy, no observability.	W-21, W-22	NFR-06 to NFR-14
G-N09	Documentation asserts capabilities the code does not have.	W-23	AR-08

2.3 Asked for by the source document, absent from the code

Four requirements in the source material have no implementation at all: the secure element that holds each device's key; the mutual token exchange that lets two devices identify each other; the online model fallback for the assistant; and the end-of-day reminder that prompts a stock review. A fifth — a real proximity radio — exists only as a same-browser channel. These are the items a reviewer will look for first, because they are the ones the concept rests on.

2.4 Baseline defects carried on the main branch

`npx tsc --noEmit` fails with two errors: `backend/src/server.ts` imports `cors`, which is not a dependency, and `backend/src/controllers/vault.controller.ts` calls the melt use case without the `userId` its DTO requires. The domain verification script passes, but only when invoked with a loader flag and experimental type stripping, which means it is not a test suite in any useful sense. Both facts are recorded rather than fixed here, because this document changes no application behaviour.

2.5 What the source material leaves open

Page 5 of the source PDF is an embedded screenshot with no text layer, so its content is unrepresented in this baseline. The strategy discussion on pages 6 and 7 (B2B protocol versus closed-loop ecosystem) is treated as V2 positioning, not as V1 scope. Currency handling is the largest silent gap in the source material: it never states which currency PayPal would actually settle in, and as section 4 shows, it cannot be naira.

3. The PRD

3.1 Product definition

T-Pay is an offline-first payment platform for markets where connectivity is intermittent and payment terminals fail during outages. Both parties to a transaction run T-Pay. Each device holds a cryptographic identity and a signing key. A payment is authorised, signed and exchanged device to device with no network present. The receiver sees a pending credit immediately. Whichever device regains connectivity first submits the signed voucher for settlement, and both devices learn the outcome.

Three properties define the product, and each is a requirement rather than a slogan:

1. **The payer cannot spend money twice.** A vault locked against the payer's PayPal balance bounds offline spending, and the bound is enforced by PayPal, not only by our ledger.
2. **A voucher cannot be forged or replayed.** Signatures bind device, vault, amount, sequence and expiry.
3. **Settlement is eventually certain.** Every voucher reaches a terminal state, and no voucher is settled twice.

3.2 Actors

ACTOR	DESCRIPTION	PRIMARY NEED
A1 Trader	Market seller with stock, often low literacy	Record a sale by voice, know what is left, reconcile at day end
A2 Salaried user	Budgets by category, pays in shops	Have a payment land in the right budget without bookkeeping
A3 Driver or conductor	Collects fares from a fixed number of seats	Know who has paid and how many seats remain
A4 Settlement service	Backend process that holds PayPal credentials and settles vouchers	Never double-settle, never lose a voucher, always be able to explain an outcome

A4 is listed as an actor deliberately. In the current codebase settlement is an activity of the client, which is why the platform's most security-sensitive behaviour is running in the least trusted place.

3.3 Scope

In V1: device identity and key custody; vault locking against a PayPal authorisation; signed offline vouchers with replay defence; server-side idempotent settlement with webhooks; dual-device ledger; the three resource-tracking use cases; offline intent parsing with an online fallback; spoken readout; a two-device demo that works with networking disabled.

Parked: see 1.11. The list is normative: nothing in it may be started before every V1 requirement below is verified.

3.4 Functional requirements

Each requirement states a testable acceptance criterion. "Testable" means a reviewer can execute the check without reading the implementation. Requirements link to the register items they close.

3.4.1 Device identity and keys

ID	REQUIREMENT	ACCEPTANCE CRITERION	CLOSES
FR-01	On first run a device generates an ECDSA P-256 keypair with a non-extractable private key.	Fresh profile: a keypair is created once and reused across restarts; attempting to export the private key fails with a platform error, not a silent success.	W-08
FR-02	The device registers its public key with the backend and receives a device token bound to it.	Registration returns a token and a key identifier; a second registration with the same key returns the existing identity rather than a duplicate.	W-08
FR-03	Every voucher names the sending key identifier, and the backend resolves it through a registry that supports revocation.	A voucher signed with a revoked key is rejected with a distinct, surfaced error, and the device is told to re-register.	W-08, G-F01
FR-04	The assurance tier of the signing key is recorded per device and shown in the interface.	A web device displays a software-key badge; a device with a hardware key displays a hardware badge; the two are never conflated in UI copy or logs.	W-08, NFR-02
FR-05	Device identity survives application restart but not credential reset.	After a restart the device signs with the same key identifier; after clearing site data it re-registers and the old key is revoked.	W-08

3.4.2 Vault

ID	REQUIREMENT	ACCEPTANCE CRITERION	CLOSES
FR-06	Locking a vault creates a PayPal order with `intent=AUTHORIZE`, has the payer approve it, and authorises it, producing an authorisation identifier stored with the vault.	Locking shows a real PayPal approval step; the stored identifier resolves through the PayPal API; no identifier is generated locally.	W-12, G-F05
FR-07	A voucher may only be signed if its amount is within the vault's remaining capacity.	Attempting to sign a voucher larger than the remaining capacity is refused before any signature is produced, with the remaining amount stated.	W-11, G-F04
FR-08	The vault tracks reserved, settled and remaining capacity separately, and remaining equals authorised minus reserved minus settled.	After two vouchers and one settlement, the displayed three figures reconcile arithmetically; a property test over random sequences finds no state where their sum exceeds the authorised amount.	W-11
FR-09	Melting a vault voids the PayPal authorisation and is refused while signed vouchers are unsettled.	Melt with a pending voucher returns a refusal naming the blocking voucher; melt with none voids the authorisation and PayPal shows the hold released.	W-12
FR-10	Every voucher records the FX rate and settlement-currency amount used at issue, obtained through a single FX port.	The voucher shows both the NGN amount and the settlement amount plus rate; the adapter contains no numeric rate literal; changing the rate source changes stored rates without code edits.	W-14, G-F08

3.4.3 Offline voucher

ID	REQUIREMENT	ACCEPTANCE CRITERION	CLOSES
FR-11	A voucher is signed over JCS canonical bytes with a domain-separation prefix, never over `JSON.stringify` output.	A known voucher verifies against a fixed byte fixture; reordering JSON keys does not change the signature; a signature produced over non-canonical bytes is rejected.	W-09, G-F02
FR-12	Amounts travel as integer minor units or decimal strings, never floats.	A voucher for 12.50 carries 1250 or "12.50"; no floating-point value appears anywhere in a signed payload.	W-09

ID	REQUIREMENT	ACCEPTANCE CRITERION	CLOSES
FR-13	Each voucher carries a monotonic sequence per device and vault, persisted before the voucher is issued.	Signing two vouchers yields consecutive sequences; restarting the app mid-flight never reissues a used sequence.	W-10, G-F03
FR-14	The backend rejects any voucher whose sequence is at or below its high-water mark for that device and vault.	Replaying an older voucher returns a replay rejection and raises an alert; the legitimate voucher is unaffected.	W-10, G-F03
FR-15	Default behaviour is offline-first: a payment is created and delivered locally first, and settles in the background if a network happens to be present.	With networking on, a payment shows as delivered instantly and settles within seconds without a user action; with networking off it behaves identically apart from the settlement timing.	W-13
FR-16	A voucher's terminal failure states are explicit and require a decision rather than a silent retry.	An expired authorisation surfaces a blocked state naming the cause; an invalid signature surfaces a rejection; neither retries indefinitely.	W-13, W-16

3.4.4 Settlement

ID	REQUIREMENT	ACCEPTANCE CRITERION	CLOSES
FR-17	Settlement runs only on the backend, and no client ever holds a payment credential.	Searching the built client bundle finds no client identifier, secret, or token endpoint; all settlement requests originate from the backend.	W-13, G-F06
FR-18	Settlement is idempotent keyed on voucher identity, with a lease so only one worker processes a voucher at a time.	Submitting the same voucher from two devices concurrently produces exactly one capture and one settled ledger entry; the loser receives the settled result rather than an error.	W-13, G-F07
FR-19	A retryable failure is retried with exponential backoff and jitter, and an ambiguous failure is reconciled by reading the resource before any retry.	A forced timeout followed by a successful read-back results in one capture, not two; retries never occur faster than the configured backoff.	W-13, G-F07
FR-20	Every settled voucher stores the PayPal order and capture identifiers and reaches exactly one terminal state.	Each settled ledger line opens to its capture identifier; no voucher remains non-terminal after a test run completes.	W-13, G-N03
FR-21	Webhook events are signature-verified, idempotent, and update voucher state for completion, pending, decline, refund, reversal and authorisation void.	Replaying the same webhook twice changes nothing the second time; a decline in either PayPal naming variant moves the voucher to a failed state.	W-13

3.4.5 Sync and ledger

ID	REQUIREMENT	ACCEPTANCE CRITERION	CLOSES
FR-22	Both devices persist the same voucher, with payer and payee views.	After an offline payment both devices list the voucher; the payer sees a pending debit and the payee a pending credit.	W-15, G-F09
FR-23	Only the backend may declare a voucher settled, and the outcome propagates to both devices.	Settling on device A updates device B's entry without a manual refresh once it reconnects.	W-15, G-F09
FR-24	Reconnection triggers settlement automatically; there is no user action to perform.	Restoring networking settles pending vouchers within the configured window with no taps.	W-16, G-F10

ID	REQUIREMENT	ACCEPTANCE CRITERION	CLOSES
FR-25	The sync pass reconciles the vault against the ledger and flags any inconsistency.	A deliberately injected over-spend is detected and reported with both figures, and the device's vault is blocked from further signing.	W-16

3.4.6 Resource tracker

ID	REQUIREMENT	ACCEPTANCE CRITERION	CLOSES
FR-26	A spoken sale deducts inventory by derivation: amount divided by registered unit price.	"Collect one thousand for coke" at ₦500 deducts exactly two units and states the resulting stock.	W-03
FR-27	A spoken inventory statement creates catalogue rows and reads them back for confirmation.	A three-item utterance creates three items and speaks a matching summary.	W-03
FR-28	A budget expense is categorised, and an unknown item raises a clarification question with one-tap answers.	An unknown item produces a question naming the candidate categories; selecting one records the expense in that category.	W-04
FR-29	Fare collection fills the seven-seat manifest and reports paid and remaining seats aloud.	Collecting fares fills seats up to capacity, never overbooks, and speaks the remaining count.	W-05
FR-30	A day-end review is produced and prompted at end of day, covering sales, payments and restock alerts.	With the schedule enabled, a review is offered at the configured hour and lists the day's totals by payment method.	W-03, G-N08

3.4.7 Assistant

ID	REQUIREMENT	ACCEPTANCE CRITERION	CLOSES
FR-31	Intent parsing works with no network and is labelled as rule-based where it is rule-based.	With networking off, all five supported intents resolve; the interface does not claim a language model where none runs.	W-21, G-F11
FR-32	When a network is present, unrecognised utterances may be sent to a server-side model, and the result is validated before it can move money.	An ambiguous utterance parses online; a model response that would issue a payment above the vault capacity is rejected before signing.	W-21, G-F11
FR-33	User-facing strings and spoken text come from a catalogue covering English, Nigerian Pidgin, Yoruba, Igbo and Hausa.	Switching language changes every visible and spoken string with no untranslated fallback left in the interface.	W-21
FR-34	Speech output degrades gracefully when no suitable voice is installed.	With speech synthesis unavailable, the confirmation is still visible and the failure is logged, not thrown as an error.	W-21

3.4.8 Client architecture

ID	REQUIREMENT	ACCEPTANCE CRITERION	CLOSES
FR-35	All surfaces read domain state through the shared core; no screen holds a private mirror of a domain fact.	Changing a vault amount in one component updates every other view without a reload; a grep for balance or vault state in component files finds no local copies.	W-17, W-18, G-F12

3.5 Non-functional requirements

Targets are numeric because a requirement without a number cannot fail.

ID	REQUIREMENT	TARGET	VERIFICATION
NFR-01	Durable offline storage	Zero accepted payments lost across a process kill, a browser restart and a 24-hour background window	Kill the process mid-payment in an automated test; the voucher is present and settleable afterwards
NFR-02	Key and secret handling	Private keys non-extractable or hardware-held; payment secrets exist only in server environment variables	Attempt key export; grep the client bundle for secrets in the build step
NFR-03	Auditability	Every settled amount traces to a capture identifier and a signing key identifier	Ledger export contains both for every settled line
NFR-04	Settlement correctness	Zero duplicate captures across 100 concurrent duplicate submissions	Concurrency test against a stubbed gateway counting captures
NFR-05	Offline transaction latency	Spoken input to on-screen and spoken confirmation within 400 ms on a mid-range Android device	Instrument the path; record on a real device, not a desktop browser
NFR-06	Speech and accessibility	Every screen usable without reading: each confirmation has a spoken form, and every control has a 44 px minimum touch target	Manual audit on a device with speech disabled and enabled
NFR-07	Localisation completeness	Zero untranslated strings in the five supported languages	Catalogue lint in CI
NFR-08	Privacy	No account number, name or amount stored unencrypted at rest; a documented retention period for settled vouchers	Inspect the storage adapter and the written policy
NFR-09	Observability	Every voucher transition logged with correlation identifier; settlement failures alertable	Log inspection after a forced failure
NFR-10	Deployment	Backend deployable from a single command with configuration from environment; health endpoint reports dependency reachability	Deploy to a clean environment and hit health
NFR-11	Type safety	Typecheck passes with no errors and is a required gate	Continuous integration run
NFR-12	Test pyramid	Domain unit tests per aggregate, an integration test per use case, one end-to-end two-device test	Continuous integration run and coverage report
NFR-13	Portability	The domain package builds for browser, React Native and Node with no platform globals below the adapter layer	Boundary check plus a build of all three targets
NFR-14	Performance headroom	Cold start under 3 s on mid-range hardware; document build time under 5 s	Measured on device and in build timing

3.6 Transaction lifecycle

A voucher has exactly one path to a terminal state. The important property is that nothing in the middle can loop forever, and nothing terminal can be re-entered.

```
DRAFT
  |  sign (key available, vault capacity sufficient, sequence persisted)
  v
SIGNED
  |  delivered over proximity transport
  v
DELIVERED
  |  peer verifies signature and persists
  v
ACKNOWLEDGED ------------------------------+
  |  settlement worker claims lease        |
  v                                        |
SETTLING                                   |  ambiguity: read back
  |            |                           |  before deciding
  |            +---------------------------+
  |  capture / partial capture
  +--> SETTLED                (terminal, capture id recorded)
  +--> RETRYABLE_FAILURE      (backoff, returns to SETTLING)
  +--> SETTLEMENT_BLOCKED     (terminal until payer re-approves)
  +--> REJECTED_REPLAY        (terminal, alert raised)
```

TRANSITION	GUARD	ON FAILURE
DRAFT to SIGNED	Vault capacity sufficient; sequence reserved	Refuse and state the remaining capacity
SIGNED to DELIVERED	Transport reports delivery	Remain SIGNED and retry delivery
DELIVERED to ACKNOWLEDGED	Signature verifies; sequence above high-water mark	REJECTED_REPLAY
ACKNOWLEDGED to SETTLING	Worker holds an exclusive lease	Leave queued
SETTLING to SETTLED	PayPal reports completion for this voucher	Record identifier, propagate to both devices
SETTLING to RETRYABLE_FAILURE	Timeout, 429, or 5xx after read-back shows no capture	Exponential backoff with jitter
SETTLING to SETTLEMENT_BLOCKED	Authorisation expired or declined	Surface to payer, require re-approval
SETTLING to REJECTED_REPLAY	Duplicate or stale sequence detected	Alert and block the device's vault

3.7 Minimum interface surface

Only what the two clients need. Every mutating endpoint is authenticated by device token and idempotent.

METHOD AND PATH	PURPOSE	IDEMPOTENCY
POST /devices/register	Register a public key, return device token and key identifier	Idempotent on public key
POST /vaults	Create and authorise a vault, storing the PayPal authorisation identifier	One active vault per user per currency
GET /vaults/current	Read authorised, reserved, settled and remaining capacity	Read only
POST /vouchers	Submit a signed voucher for acceptance and settlement	Keyed on voucher identity
GET /vouchers	List vouchers for the calling device with payer or payee view	Read only

METHOD AND PATH	PURPOSE	IDEMPOTENCY
GET /vouchers/{id}	Read one voucher including capture identifier	Read only
POST /vaults/{id}/melt	Void the authorisation and return remaining capacity to the payer	Refused while vouchers are unsettled
POST /webhooks/paypal	Receive PayPal events, verified and idempotent	Keyed on event identifier
GET /health	Report liveness and dependency reachability	Read only

3.8 Definition of done for V1

A reviewer with two devices and no cooperation from the build team must be able to:

1. Install or open the app on both devices and see each device's identity and assurance tier.
2. Lock a vault while online, completing a real PayPal approval, and see the authorisation reflected back from PayPal.
3. Disable networking on both devices and make a payment; the payer sees a pending debit and the payee a pending credit within a second.
4. Attempt a second payment larger than the remaining capacity and be refused before a signature exists.
5. Re-enable networking on one device only and watch the voucher settle without any user action.
6. Confirm both devices show the settled state, each with the same capture identifier.
7. Melt the vault and confirm the remaining hold is released at PayPal, not merely in the app.
8. Replay a captured voucher at the API and see it rejected as a replay.
9. Take a spoken sale offline and see stock deducted by derivation, with the confirmation spoken aloud.
10. Run the test suite and the typecheck from a clean clone and see both pass.

4. PayPal constraints as requirements

PayPal is not a neutral rail. It constrains the product in ways the concept has to absorb, and several of those constraints are load-bearing: the currency model and the authorisation model each change the design, not just the implementation.

4.1 What PayPal will and will not do

PayPal Orders v2 accepts a fixed list of currency codes: AUD, BRL, CAD, CNY, CZK, DKK, EUR, HKD, HUF, ILS, JPY, MYR, MXN, TWD, NZD, NOK, PHP, PLN, GBP, SGD, SEK, CHF, THB and USD. **NGN is not among them**; a request carrying it fails with `CURRENCY_NOT_ALLOWED`. HUF, JPY and TWD take no decimal places; everything else expects two.

The consequence is unavoidable and better stated plainly than discovered late: a naira amount cannot be held or charged by PayPal. V1 keeps NGN as the currency the user sees and reasons about, and settles in a supported currency at a rate snapshotted when the voucher is signed. That rate is part of the signed payload, so a voucher cannot change value between devices or between issue and settlement.

The second useful discovery is that PayPal's authorisation model maps onto the vault almost exactly. An order created with `intent=AUTHORIZE`, then authorised, puts a hold on the payer's funds; that authorisation can be captured **partially and repeatedly** while `final_capture` remains false; and the remainder is released by voiding the authorisation. Lock, spend, melt therefore become create-and-authorise, partial capture, void — with PayPal itself enforcing the ceiling rather than trusting our arithmetic.

The caveats are real: an authorisation's honor period is three days and its validity twenty-nine, after which a reauthorisation (day four onwards) yields a new identifier and a fresh honor period. A voucher whose authorisation has expired must therefore become a blocked state requiring the payer to re-approve, not a retry that spins forever.

4.2 Requirements

ID	REQUIREMENT	ACCEPTANCE CRITERION	SOURCE
PP-01	Never send NGN to PayPal. All amounts crossing the API use a supported currency code with the correct decimal precision, converted through the FX port with the rate recorded on the voucher.	Requesting any PayPal operation with NGN fails before the call is made; a test asserts the currency is in the allowed set and the value matches the required precision.	developer.paypal.com/reference/currency-codes
PP-02	Client identifier and secret exist only in backend environment variables. Clients receive only an identifier where the SDK requires one, never a secret.	Bundle search finds no secret; rotating the secret requires no client release.	developer.paypal.com/api/rest/authentication/

ID	REQUIREMENT	ACCEPTANCE CRITERION	SOURCE
PP-03	The OAuth token is cached in memory and refreshed before `expires_in` elapses; one token per settlement is a defect.	A batch of ten settlements produces one token request.	developer.paypal.com/api/rest/authentication/
PP-04	The vault is an authorisation: create with `intent=AUTHORIZE`, authorise, then capture partially with `final_capture=false`.	Each settled voucher corresponds to a capture against the vault's authorisation identifier; the sum of captures never exceeds the authorised amount.	developer.paypal.com/v5/checkout/auth-capture
PP-05	Authorisation lifetime is respected: capture inside the three-day honor period, reauthorise from day four while the authorisation is valid, and treat expiry as a blocking state that requires payer action.	A simulated expired authorisation produces SETTLEMENT_BLOCKED, not a retry loop; the reauthorisation job only runs inside the allowed window.	developer.paypal.com/api/payments/v2/authorizations-reauthorize
PP-06	Own the deduplication. The database is the authority keyed on voucher identity; `PayPal-Request-Id` is a UUID bound to the payload hash for a single logical attempt. Assume PayPal remembers it for six hours at most.	A duplicate submitted eight hours later is still deduplicated by our store; the same key is never reused with a different body.	developer.paypal.com/api/rest/reference/idempotency
PP-07	Reconcile before retry. On timeout or 5xx, read the order or authorisation back before deciding; retry only timeouts and 5xx, with backoff and jitter, serialised per voucher.	A forced timeout with a completed capture results in one capture and a settled voucher.	developer.paypal.com/api/make-api-requests
PP-08	Webhooks are the source of truth; polling is a bounded reconciliation fallback. Handlers verify signatures, are idempotent, and accept both decline spellings.	Replaying an event is a no-op; both `PAYMENT.CAPTURE.DECLINED` and `PAYMENT.CAPTURE.DENIED` are handled.	developer.paypal.com/api/rest/webhooks/event-names
PP-09	Respect rate limiting: no polling loops, serialised writes per voucher, backoff on 429.	A 429 in a stubbed gateway delays the next attempt by the configured backoff.	developer.paypal.com/api/rest/reference/rate-limiting

ID	REQUIREMENT	ACCEPTANCE CRITERION	SOURCE
PP-10	Sandbox only in V1, against the sandbox base URL, with an explicit refusal to run live mode.	Live configuration is rejected at startup with a clear message.	developer.paypal.com/api/rest/integration/orders-api
PP-11	Handled events are exactly: capture completed, pending, declined, denied, refunded, reversed; authorisation voided; and payment-approval reversed.	Each event maps to a documented voucher transition or is explicitly ignored with a log line.	developer.paypal.com/api/rest/webhooks/event-names
PP-12	Settlement failures are classified into retryable, blocked and rejected classes, because conflating them is what produces both infinite retries and silently lost money.	Each class has a test asserting the resulting voucher state and the operator-visible message.	developer.paypal.com/api/rest/integration/payments-api/v1-v2-migration

4.3 Consequences the product must accept

- **The vault holds a supported currency, not naira.** Displaying ₦10,000 against a USD hold requires a rate; that rate is signed into every voucher, and a rate change between lock and spend is visible rather than silent.
- **Authorisation is not a debit.** Money is held, not taken. The melt operation is a void, and calling it a refund would misdescribe the system.
- **Hold validity bounds offline life.** A voucher older than the authorisation's remaining validity cannot settle. The product therefore needs a stated maximum offline window, enforced at signing time.
- **A sandbox merchant account may not be able to receive every supported currency.** This is unverified and belongs in the open decisions list; it can reduce the demo to USD only.

4.4 Sources

TOPIC	URL
Currency codes	https://developer.paypal.com/reference/currency-codes
Currency error	https://developer.paypal.com/api/errors/currency-not-allowed
Idempotency	https://developer.paypal.com/api/rest/reference/idempotency
Making API requests	https://developer.paypal.com/api/make-api-requests
Orders API integration	https://developer.paypal.com/api/rest/integration/orders-api
Authorise then capture	https://developer.paypal.com/v5/checkout/auth-capture
Reauthorise	https://developer.paypal.com/api/payments/v2/authorizations-reauthorize
Capture an authorisation	https://developer.paypal.com/api/payments/v2/authorizations-capture
Webhook event names	https://developer.paypal.com/api/rest/webhooks/event-names

TOPIC	URL
Authentication	https://developer.paypal.com/api/rest/authentication/
Rate limiting	https://developer.paypal.com/api/rest/reference/rate-limiting
v1 to v2 migration	https://developer.paypal.com/api/rest/integration/payments-api/v1-v2-migration
Orders v2 OpenAPI specification	https://raw.githubusercontent.com/paypal/paypal-rest-api-specifications/main/openapi/checkout_orders_v2.json

5. Device trust

The concept calls for a secure enclave on both devices. What is achievable differs by platform, and saying so is part of the requirement.

5.1 Two assurance tiers

TIER	PLATFORMS	WHAT IT GIVES	WHAT IT DOES NOT GIVE
A — hardware-backed	Android Keystore with StrongBox; iOS Secure Enclave	Private key never enters application memory; signing is a hardware operation; optional attestation	Nothing about the honesty of the amount being signed
B — software-protected	Web: non-extractable WebCrypto key in IndexedDB; Expo: software key with the secret wrapped in the platform secure store	Key cannot be read back through the API; a copy of storage alone is not enough to sign	A determined attacker with device control can sign; the specification itself does not promise hardware protection

The WebCrypto specification describes how keys may be stored by existing storage mechanisms and explicitly declines to require that key material stay off disk. Non-extractable is a strong API-level guarantee and a weak device-level one. Tier B is acceptable for V1 and must be labelled as such wherever assurance is claimed.

Expo SDK 52 cannot produce a hardware signing key without a native module: `expo-secure-store` stores strings, `react-native-keychain` stores credentials, and `expo-crypto` hashes and encrypts but does not sign asymmetrically. Reaching Tier A on mobile is a native-module task, scheduled in M7, not an app-configuration change.

5.2 Signing rules

ID	RULE
DT-01	ECDSA over P-256 with SHA-256 is the single algorithm for V1. It is available in WebCrypto, in Node and on both mobile platforms, which Ed25519 is not yet.
DT-02	Payloads are canonicalised with JCS (RFC 8785). Never sign `JSON.stringify` output: key order, number formatting and unrelated whitespace differ between JavaScript engines.
DT-03	Signatures are transmitted as base64url of the raw 64-byte `r‖s` pair and normalised at the boundary, because WebCrypto emits raw while Node and Java emit DER.
DT-04	A domain-separation prefix is prepended to the canonical bytes, so a signature over a voucher can never be replayed as a signature over something else.
DT-05	Amounts are integer minor units or decimal strings. Floating-point values never enter a signed payload.
DT-06	The payload binds key identifier, device identifier, vault identifier, sequence, amount, settlement amount, FX rate, payee, issue time and expiry.
DT-07	Replay defence is layered: monotonic sequence per device and vault, persisted before issue; a server high-water mark; a nonce cache; and a short expiry. No single layer is trusted alone.
DT-08	Double-spend detection is online reconciliation: a batch whose totals exceed the authorisation, or whose sequence sits below the high-water mark, blocks that device's vault and raises an alert.

5.3 Why the layering matters

The Bank of England's 2025 digital pound offline-payments experiment reached the same conclusion from the other direction: vendors relied on secure elements for key protection, and all four solutions used online reconciliation as the fallback that detects counterfeiting and double spending. The report is explicit that a breached secure element would permit both. Our design follows that finding rather than assuming hardware alone is sufficient.

5.4 Sources

TOPIC	URL
WebCrypto key storage	https://w3c.github.io/webcrypto/
CryptoKey.extractable	https://developer.mozilla.org/en-US/docs/Web/API/CryptoKey/extractable
Digital signature guidance	https://developer.chrome.com/docs/modern-web-guidance/guides/security/digital-signatures
JCS, RFC 8785	https://www.rfc-editor.org/rfc/rfc8785.html
Android Keystore	https://developer.android.com/privacy-and-security/keystore
Expo SecureStore	https://docs.expo.dev/versions/latest/sdk/securestore/
react-native-keychain	https://oblador.github.io/react-native-keychain/docs/platform-value-storage/
Offline payments experiment	https://www.bankofengland.co.uk/report/2025/digital-pound-experiment-report-offline-payments
GNU Taler double-spend detection	https://www.taler.net/en/news/2025-01.html

6. Architecture contract

These are constraints, not preferences. Each rule names the specific place in the current codebase that violates it, because a rule that has never been broken is a rule nobody needs.

6.1 Rules

ID	RULE	RATIONALE	CURRENT VIOLATION
AR-01	Dependencies point inward only: domain depends on nothing outside the shared kernel; application depends on domain and ports; infrastructure depends on application and domain; presentation depends on application.	A domain that imports a framework cannot be tested, reused or reasoned about.	None at present; the violation arrives if the new storage port is skipped in W-19.
AR-02	No platform global below the adapter layer. `window`, `navigator`, `document`, `fetch`, `localStorage` and `fs` may appear only inside an infrastructure adapter.	The same domain must run in a browser, in React Native and in Node.	`PayPalSdkGatewayAdapter` reads `navigator.onLine` and `localStorage`; every `LocalStorage*Repository` touches `window`.
AR-03	Single ownership of state. A domain fact has exactly one owner; a view reads it and never mirrors it.	Two copies of a balance is two answers to the same question.	`PayPalEscrowModal` keeps its own vault amount, spent amount and token list beside the domain.
AR-04	One implementation per behaviour.	Duplicated logic diverges silently.	`VoiceAssistantModal` contains an inline intent parser beside `LocalSLMAdapter`.
AR-05	Every external edge is a port: network, storage, cryptography, clock, randomness, speech, proximity. Aggregates never read the wall clock or generate randomness directly.	Non-deterministic aggregates cannot be tested and cannot be replayed.	`MarketVault.create` and `CryptographicExchangeService` call `new Date()` and `Math.random()` inline.
AR-06	Only the composition root constructs adapters, and nothing reaches a singleton from inside the domain.	Wiring in one place is what makes substitution possible.	Hooks and components reach the global container directly.
AR-07	No speculative code. Unused exports, entities with no use case and flags for the future are deleted rather than kept.	Unused code is untested code that still has to be maintained.	`CryptographicExchangeService.verifyProof` is never called; parts of the escrow event set are never published.
AR-08	Claims must be executable. A capability may be described as working only if a test or a reachable UI path exercises it.	The gap between README and behaviour is itself a defect.	`README.md` asserts signatures, mesh networking, a secure enclave and a working PayPal integration.
AR-09	Every mutating API is authenticated by device token and idempotent.	A retried request must not be a second payment.	The REST layer accepts any caller as any user.

6.2 Bounded context ownership

CONTEXT	OWNS	MAY IMPORT	MUST NEVER TOUCH
Shared kernel	Identity, money, result, aggregate base, event bus	Nothing	Any context
Escrow	Vault, voucher, cryptogram, settlement state	Shared kernel	Tracker or assistant internals

CONTEXT	OWNS	MAY IMPORT	MUST NEVER TOUCH
Merchant	Inventory, sale, price derivation, reconciliation	Shared kernel	Vault or settlement internals
Budget	Plan, categories, expenses, ambiguity	Shared kernel	Vault or settlement internals
Transit	Trip, seats, fares	Shared kernel, proximity port only	Vault or settlement internals
Assistant	Intent, language, speech	Shared kernel and application use cases	Any aggregate internals directly

Cross-context communication is domain events only. No context may read another's aggregates.

6.3 Import matrix

FROM → TO	SHARED	DOMAIN	APPLICATION	INFRASTRUCTURE	PRESENTATION
Shared	Yes	No	No	No	No
Domain	Yes	Same context	No	No	No
Application	Yes	Yes	Same context	Ports only	No
Infrastructure	Yes	Yes	Yes	Same platform	No
Presentation	Types only	No	Yes	No	Same layer

6.4 Review checklist

Run before any change is accepted, in this order. A "yes" answer needs a justification in the pull request.

1. Does it add an import across a boundary in 6.3? (AR-01)
2. Does it introduce a platform global outside an adapter? (AR-02)
3. Does it create a second holder of a fact that already has an owner? (AR-03)
4. Does it duplicate behaviour that exists elsewhere? (AR-04)
5. Does it read the clock or randomness outside a port? (AR-05)
6. Does it construct an adapter outside the composition root? (AR-06)
7. Does it add an export nothing calls, a flag nothing reads, or a type nothing uses? (AR-07)
8. Does it claim a capability that no test or reachable path exercises? (AR-08)
9. Does it add a mutating endpoint without authentication and idempotency? (AR-09)

6.5 Enforcement

Rules that rely on memory are not rules. Three automated gates, added in M0 and kept green:

GATE	MECHANISM	CATCHES
Import boundaries	Static rule file checked in CI, expressing the matrix in 6.3	AR-01, AR-05, AR-06
Forbidden globals	Grep gate over everything outside `infrastructure/` adapters	AR-02
Unused exports	Dead-code check on the domain and application layers	AR-07

7. Roadmap

Ordered by dependency, not by enthusiasm. Each milestone states the outcome a reviewer can observe, not the work performed.

ID	MILESTONE	OUTCOME	EXIT CRITERIA	CLOSES
M0	Conformance baseline	The repository is honest and clean before new work lands	Typecheck passes; three architecture gates run in CI; README matches reality; verification script runs under a real runner	W-20, W-23, G-N06
M1	Shared core package	One domain, three surfaces	The core builds for browser, React Native and Node; no platform globals below adapters; clock and randomness ports exist	W-19, G-N07
M2	Device identity and keys	Each device can prove who it is	Keypair generated and persisted; registration and device token issued; revoked keys rejected; assurance tier displayed	W-08, G-F01
M3	Signed vouchers	A voucher is tamper-evident and unrepeatable	JCS signing verified against fixtures; sequence and nonce enforced; replay rejected	W-09, W-10, G-F02, G-F03
M4	Vault as authorisation	PayPal enforces the ceiling	Lock creates a real authorisation; partial captures respect capacity; melt voids; reserved and settled reconcile	W-11, W-12, W-14, G-F04, G-F05, G-F08
M5	Server-side settlement	Money moves in one place, once	Idempotent capture with lease; webhooks handled; retry classification; capture identifiers recorded	W-13, G-F06, G-F07, G-N03, G-N04
M6	Dual-device ledger and sync	Both devices agree, always	Payee persistence; reconnect-triggered settlement; outcome propagated; reconciliation flags over-spend	W-15, W-16, G-F09, G-F10
M7	React Native parity	The second real device	Mobile consumes the shared core; Tier A keys via native module; real proximity transport replaces the browser channel	W-18
M8	Assistant and localisation	Voice works offline, honestly	Five intents offline; online fallback validated before it can pay; catalogue-driven strings; graceful speech fallback	W-21, G-F11
M9	Test pyramid and durability	Failures are caught before users	Domain unit tests, per-use-case integration tests, one two-device end-to-end test, durable storage with kill-test	W-22, G-N01, G-N05
M10	Demo hardening	A judge can verify the claims unaided	Definition of done in 3.8 executed end to end on two devices with networking disabled; ledger export available	All

Sequencing is strict in one place: M2 before M3, M3 before M4, M4 before M5, M5 before M6. Everything else may overlap.

7.1 Parked

The list in 1.11 is normative for the whole roadmap. No parked item may be started, scoped or prototyped before M10 is signed off.

8. Open inputs and decisions

ID	OPEN ITEM	WHY IT MATTERS	OWNER DECISION NEEDED
O-01	Page 5 of the source PDF is an image with no text layer	Its content may add or contradict requirements	Provide the text or a description
O-02	Whether the sandbox merchant account can receive the intended settlement currency	If not, the demo is USD-only and the FX story narrows	Confirm by attempting a sandbox capture
O-03	Which assurance tier the demo presents	Tier B is acceptable but must be labelled; presenting it as hardware would be false	Choose the tier to demonstrate
O-04	Source of the FX rate, and whether it is fixed for the demo	A signed rate implies an audit trail for where it came from	Choose a source, or accept a fixed demo rate labelled as such
O-05	Maximum offline window	Bounded by authorisation validity and by product intent	Set the limit (proposed: 24 hours)
O-06	Whether the web build remains a supported surface after M7	Web cannot reach Tier A keys, so it is a demo surface, not a production one	Decide the web surface's role

Appendix A. Evidence index

PATH	SUPPORTS
`core/shared/domain/`	W-01
`core/escrow/domain/`	W-02, W-06, W-11
`core/escrow/application/ports/`	W-02
`core/escrow/domain/services/CryptographicExchangeService.ts`	W-09
`core/escrow/domain/value-objects/DeviceIdentity.ts`	W-08
`core/escrow/infrastructure/adapters/PayPalSdkGatewayAdapter.ts`	W-13, W-14
`core/escrow/infrastructure/adapters/BroadcastChannelProximityAdapter.ts`	W-15
`core/escrow/infrastructure/repositories/LocalStorageEscrowRepository.ts`	W-19
`core/resource-tracking/merchant/`	W-03
`core/resource-tracking/budget/`	W-04
`core/resource-tracking/transit/`	W-05
`core/hybrid-assistant/infrastructure/adapters/LocalSLMAdapter.ts`	W-21
`core/container.ts`	W-07, W-15
`core/__tests__/domain-verification.ts`	W-07, W-11
`src/components/PayPalEscrowModal.tsx`	W-17
`src/components/VoiceAssistantModal.tsx`	W-17, W-21
`src/components/views/TrackerView.tsx`	W-17
`src/hooks/useTPayDomain.ts`	W-16
`react-native-app/App.tsx`	W-18
`backend/src/server.ts`	W-20
`backend/src/controllers/vault.controller.ts`	W-20
`README.md`	W-23
`DDD_ARCHITECTURE.md`	W-23

Appendix B. Trace from the source document

SOURCE MATERIAL	PRODUCES	STATUS TODAY
Hybrid assistant that works online and offline (p1)	FR-31, FR-32	Simulated
Lightweight on-device model, any technology (p1)	FR-31	Simulated
Resource tracker tracking stock, budget or seats by use case (p1)	FR-26 to FR-29	Met in domain, Partial in UI
Three demo cases: trader, salaried user, bus driver (p1)	FR-26 to FR-29	Met in domain
Spoken sale with price-to-quantity derivation (p1)	FR-26	Met
Budget deduction with category and clarification (p2)	FR-28	Met in domain
Seven-seat korope with collect-from-everybody (p2)	FR-29	Met in domain
Spoken inventory setup with read-back in a chosen language (p2)	FR-27, FR-33	Met in domain, Simulated in UI
End-of-day stock review and reminder (p2)	FR-30	Missing
Both devices exchange a token identifying each other (p3)	FR-01, FR-02, FR-11, FR-22	Missing
Whichever device reconnects first reports to PayPal (p3)	FR-23, FR-24	Simulated
Offline-first default, settle silently when online (p3)	FR-15	Partial
Funds locked in an offline wallet for a period and returned if unused (p3)	FR-06, FR-09	Simulated
PayPal as primary processor, SDK and API requirements (p1, p3)	PP-01 to PP-12	Simulated
Offline protocol as an API for banks (p4, p6, p7)	Parked, see 1.11	Not in V1